

TÀI LIỆU THIẾT KẾ

Dự Án Spring Boot

Social App - User / Post / Like / Comment

Phiên bản:	v1.0
Ngày tạo:	06/03/2026
Công nghệ:	Java 17 · Spring Boot 3 · Spring Data JPA · MySQL
Phạm vi:	Không bao gồm Auth & Phân quyền (giai đoạn sau)

MỤC LỤC

1. Thiết kế Cơ sở dữ liệu

- 1.1 Sơ đồ quan hệ (ERD)
- 1.2 Chi tiết các bảng

2. Kiến trúc Dự án

- 2.1 Cấu trúc thư mục
- 2.2 Luồng xử lý Request-Response

3. Danh sách API

4. DEMO chi tiết – 2 API mẫu

- 4.1 GET /api/users/{id} – Lấy thông tin người dùng
- 4.2 POST /api/posts – Tạo bài viết mới

5. Yêu cầu các API còn lại (từ Dễ → Khó)

- 5.1 Nhóm User
- 5.2 Nhóm Post
- 5.3 Nhóm Like
- 5.4 Nhóm Comment
- 5.5 API nâng cao – JOIN nhiều bảng

6. Quy ước chung & Validate

1. Thiết kế Cơ sở dữ liệu

1.1 Sơ đồ quan hệ (ERD)

Dự án gồm 4 bảng chính với quan hệ như sau:

Bảng	Quan hệ	Bảng đích	Kiểu
users	1 → N	posts	Một user có nhiều bài post
users	1 → N	likes	Một user có nhiều lượt like
users	1 → N	comments	Một user có nhiều comment
posts	1 → N	likes	Một post có nhiều lượt like
posts	1 → N	comments	Một post có nhiều comment
comments	1 → N	comments	Comment có thể có reply (self-ref)

□ Khóa ngoại: $likes(user_id) \rightarrow users(id)$, $likes(post_id) \rightarrow posts(id)$, $comments(user_id) \rightarrow users(id)$, $comments(post_id) \rightarrow posts(id)$, $comments(parent_id) \rightarrow comments(id)$, $posts(user_id) \rightarrow users(id)$.

1.2 Chi tiết các bảng

Bảng: users

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
username	VARCHAR(50)	NOT NULL, UNIQUE	Tên đăng nhập (duy nhất)
email	VARCHAR(100)	NOT NULL, UNIQUE	Email người dùng
full_name	VARCHAR(100)	NOT NULL	Họ và tên đầy đủ
avatar_url	VARCHAR(255)	NULLABLE	URL ảnh đại diện
bio	TEXT	NULLABLE	Giới thiệu bản thân
status	ENUM	DEFAULT ACTIVE	ACTIVE INACTIVE BANNED
created_at	DATETIME	NOT NULL	Thời gian tạo tài khoản
updated_at	DATETIME	NOT NULL	Thời gian cập nhật cuối

Bảng: posts

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
---------	--------------	-----------	-------

id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
user_id	BIGINT	FK → users(id)	Người đăng bài
title	VARCHAR(255)	NOT NULL	Tiêu đề bài viết
content	TEXT	NOT NULL	Nội dung bài viết
image_url	VARCHAR(255)	NULLABLE	Ảnh đính kèm (nếu có)
status	ENUM	DEFAULT PUBLISHED	PUBLISHED DRAFT DELETED
view_count	INT	DEFAULT 0	Số lượt xem
created_at	DATETIME	NOT NULL	Thời gian đăng bài
updated_at	DATETIME	NOT NULL	Thời gian chỉnh sửa cuối

Bảng: likes

□ *UNIQUE constraint: (user_id, post_id) - mỗi user chỉ like mỗi bài 1 lần.*

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
user_id	BIGINT	FK → users(id)	Người thực hiện like
post_id	BIGINT	FK → posts(id)	Bài viết được like
created_at	DATETIME	NOT NULL	Thời gian like

Bảng: comments

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
post_id	BIGINT	FK → posts(id)	Bài viết được bình luận
user_id	BIGINT	FK → users(id)	Người bình luận
parent_id	BIGINT	FK → comments(id), NULLABLE	Reply comment cha (null = root)
content	TEXT	NOT NULL	Nội dung bình luận
status	ENUM	DEFAULT VISIBLE	VISIBLE HIDDEN DELETED
created_at	DATETIME	NOT NULL	Thời gian bình luận
updated_at	DATETIME	NOT NULL	Thời gian cập nhật

SQL DDL (MySQL)

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  full_name VARCHAR(100) NOT NULL,  
  avatar_url VARCHAR(255),  
  bio TEXT,  
  status ENUM('ACTIVE', 'INACTIVE', 'BANNED') DEFAULT 'ACTIVE',  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);  
  
CREATE TABLE posts (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  content TEXT NOT NULL,  
  image_url VARCHAR(255),  
  status ENUM('PUBLISHED', 'DRAFT', 'DELETED') DEFAULT 'PUBLISHED',  
  view_count INT DEFAULT 0,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);  
  
CREATE TABLE likes (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  post_id BIGINT NOT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY uk_user_post (user_id, post_id),  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE  
);  
  
CREATE TABLE comments (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  post_id BIGINT NOT NULL,  
  user_id BIGINT NOT NULL,  
  parent_id BIGINT,  
  content TEXT NOT NULL,  
  status ENUM('VISIBLE', 'HIDDEN', 'DELETED') DEFAULT 'VISIBLE',  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
```

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
FOREIGN KEY (parent_id) REFERENCES comments(id) ON DELETE SET NULL  
);
```

2. Kiến trúc Dự án

2.1 Cấu trúc thư mục

Dự án theo chuẩn Layered Architecture (Controller → Service → Repository):

```
src/main/java/com/example/socialapp/
├─ SocialAppApplication.java # Main class
├─ config/
│   └─ AppConfig.java # Bean cấu hình chung
├─ controller/
│   ├── UserController.java
│   ├── PostController.java
│   ├── LikeController.java
│   └─ CommentController.java
├─ service/
│   ├── UserService.java # Interface
│   ├── impl/UserServiceImpl.java
│   ├── PostService.java
│   ├── impl/PostServiceImpl.java
│   ├── LikeService.java
│   ├── impl/LikeServiceImpl.java
│   ├── CommentService.java
│   └─ impl/CommentServiceImpl.java
├─ repository/
│   ├── UserRepository.java # extends JpaRepository
│   ├── PostRepository.java
│   ├── LikeRepository.java
│   └─ CommentRepository.java
├─ entity/
│   ├── User.java # @Entity
│   ├── Post.java
│   ├── Like.java
│   └─ Comment.java
├─ dto/
│   ├── request/
│   │   ├── CreateUserRequest.java
│   │   ├── updateUserRequest.java
│   │   ├── CreatePostRequest.java
│   │   ├── UpdatePostRequest.java
│   │   ├── LikeRequest.java
│   │   ├── CreateCommentRequest.java
│   │   └─ UpdateCommentRequest.java
│   └─ response/
```

	└─ UserResponse.java
	└─ UserSummaryResponse.java
	└─ PostResponse.java
	└─ PostDetailResponse.java
	└─ LikeResponse.java
	└─ CommentResponse.java
	└─ ApiResponse.java # Wrapper chung
	└─ exception/
	└─ ResourceNotFoundException.java
	└─ DuplicateResourceException.java
	└─ BadRequestException.java
	└─ GlobalExceptionHandler.java # @RestControllerAdvice
	└─ mapper/
	└─ UserMapper.java # MapStruct (hoặc thủ công)
	└─ PostMapper.java
	└─ CommentMapper.java
	└─ util/
	└─ PageableUtil.java # Helper phân trang

2.2 Luồng xử lý Request-Response

Bước	Tầng	Trách nhiệm
1	Client	Gửi HTTP Request (JSON body / query params)
2	Controller	Nhận request, gọi @Valid để validate DTO, chuyển cho Service
3	Service	Xử lý business logic, gọi Repository
4	Repository	Truy vấn DB (JPA / JPQL / Native Query)
5	Service	Nhận Entity, convert sang Response DTO
6	Controller	Wrap vào ApiResponse, trả về ResponseEntity
7	Client	Nhận HTTP Response (JSON)
△	Exception	GlobalExceptionHandler bắt mọi exception, trả về lỗi chuẩn

Wrapper Response chuẩn - ApiResponse

```

public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;
    private int statusCode;
    // Constructor, Getter, Setter

```



```
}  
  
// Ví dụ response thành công:  
{  
  "success": true,  
  "message": "Thành công",  
  "statusCode": 200,  
  "data": { ... }  
}  
  
// Ví dụ response lỗi:  
{  
  "success": false,  
  "message": "Không tìm thấy người dùng với id = 99",  
  "statusCode": 404,  
  "data": null  
}
```

3. Danh sách API Tổng quan

Tất cả API đều có prefix: /api/v1. Định dạng request/response: application/json. Phân trang sử dụng query param: ?page=0&size=10&sort=createdAt,desc

Method	Endpoint	Mô tả	Độ khó
GET	/api/v1/users	Lấy danh sách người dùng (có phân trang)	Dễ
GET	/api/v1/users/{id}	Lấy thông tin chi tiết một user	Dễ
POST	/api/v1/users	Tạo người dùng mới	Dễ
PUT	/api/v1/users/{id}	Cập nhật toàn bộ thông tin user	Dễ
PATCH	/api/v1/users/{id}/status	Thay đổi trạng thái user (ACTIVE/BANNED)	Dễ
DELETE	/api/v1/users/{id}	Xóa người dùng	Dễ
GET	/api/v1/posts	Danh sách bài viết (có phân trang + filter)	Dễ
GET	/api/v1/posts/{id}	Chi tiết bài viết (tăng view_count)	Dễ
POST	/api/v1/posts	Tạo bài viết mới	Dễ
PUT	/api/v1/posts/{id}	Cập nhật bài viết	Dễ
DELETE	/api/v1/posts/{id}	Xóa bài viết (soft delete → DELETED)	Dễ
POST	/api/v1/likes	Like / Unlike bài viết (toggle)	Trung bình
GET	/api/v1/posts/{id}/likes	Danh sách user đã like bài viết	Trung bình
GET	/api/v1/posts/{id}/comments	Danh sách comment gốc của bài viết	Trung bình
POST	/api/v1/comments	Thêm comment (có thể reply comment khác)	Trung bình
PUT	/api/v1/comments/{id}	Chỉnh sửa comment	Trung bình
DELETE	/api/v1/comments/{id}	Xóa comment (soft delete)	Trung bình

GET	/api/v1/comments/{id}/replies	Lấy danh sách reply của một comment	Trung bình
GET	/api/v1/users/{id}/posts	Bài viết của một user (JOIN posts+users)	Trung bình
GET	/api/v1/posts/search	Tìm kiếm bài viết theo keyword	Trung bình
GET	/api/v1/posts/top-liked	Top bài viết nhiều like nhất (JOIN+GROUP)	Khó
GET	/api/v1/posts/{id}/detail	Chi tiết post: tác giả + like count + comments	Khó
GET	/api/v1/users/{id}/stats	Thống kê user: post count, like nhận, comment	Khó
GET	/api/v1/feed	Feed: bài viết mới kèm thông tin user + like count	Khó
GET	/api/v1/comments/tree/{postId}	Cây comment (nested) theo bài viết	Khó
GET	/api/v1/posts/trending	Trending: điểm = like*2 + comment, 7 ngày gần	Khó

4. DEMO Chi tiết - 2 API Mẫu

Hai API dưới đây mô tả đầy đủ luồng: Controller → Service → Repository, cùng Request DTO, Response DTO, Validate và xử lý exception.

4.1 GET /api/v1/users/{id} - Lấy thông tin người dùng

Method	GET
URL	/api/v1/users/{id}
Path Param	id - BIGINT, bắt buộc
Auth	Không yêu cầu
Mô tả	Trả về thông tin chi tiết của một người dùng theo ID

Response thành công (200 OK)

```
{
  "success": true,
  "message": "Thành công",
  "statusCode": 200,
  "data": {
    "id": 1,
    "username": "nguyenvana",
    "email": "vana@example.com",
    "fullName": "Nguyễn Văn A",
    "avatarUrl": "https://cdn.example.com/avatar/1.jpg",
    "bio": "Lập trình viên yêu thích Java",
    "status": "ACTIVE",
    "createdAt": "2026-01-15T08:30:00"
  }
}
```

Response lỗi (404 Not Found)

```
{
  "success": false,
  "message": "Không tìm thấy người dùng với id = 99",
  "statusCode": 404,
  "data": null
}
```

Source Code - UserController.java

```
@RestController
```

```

@RequestMapping("/api/v1/users")
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;

    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<UserResponse>> getUserById(
        @PathVariable @Positive Long id) {
        UserResponse data = userService.getUserById(id);
        return ResponseEntity.ok(ApiResponse.success(data));
    }
}

```

Source Code - UserServiceImpl.java

```

@Service
@RequiredArgsConstructor
public class UserServiceImpl implements UserService {
    private final UserRepository userRepository;
    private final UserMapper userMapper;

    @Override
    public UserResponse getUserById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Không tìm thấy người dùng với id = " + id));
        return userMapper.toResponse(user);
    }
}

```

Source Code - UserRepository.java

```

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // findById đã có sẵn từ JpaRepository
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
    Optional<User> findByUsername(String username);
}

```

GlobalExceptionHandler (xử lý 404)

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(
        ResourceNotFoundException ex) {
    }
}

```

```
return ResponseEntity.status(404)
    .body(ApiResponse.error(ex.getMessage(), 404));
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiResponse<Map<String,String>>> handleValidation(
    MethodArgumentNotValidException ex) {
    Map<String,String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(e ->
        errors.put(e.getField(), e.getDefaultMessage()));
    return ResponseEntity.status(400)
        .body(ApiResponse.error("Dữ liệu không hợp lệ", 400, errors));
}
}
```

4.2 POST /api/v1/posts - Tạo bài viết mới

Method	POST
URL	/api/v1/posts
Content-Type	application/json
Auth	Không yêu cầu (truyền userId trong body)
Mô tả	Tạo bài viết mới cho một người dùng

Request Body - CreatePostRequest.java

```
public class CreatePostRequest {  
    @NotNull(message = "userId không được đ[] tr[]ng")  
    @Positive(message = "userId phải là s[] dương")  
    private Long userId;  
  
    @NotBlank(message = "Tiêu đề không được đ[] tr[]ng")  
    @Size(min=5, max=255, message = "Tiêu đề từ 5 đ[]n 255 ký tự")  
    private String title;  
  
    @NotBlank(message = "Nội dung không được đ[] tr[]ng")  
    @Size(min=10, message = "Nội dung t[]i thi[]u 10 ký tự")  
    private String content;  
  
    @URL(message = "imageUrl không đúng định dạng URL")  
    private String imageUrl; // optional  
  
    @Pattern(regex = "PUBLISHED|DRAFT",  
            message = "status ch[] nhận PUBLISHED hoặc DRAFT")  
    private String status = "PUBLISHED";  
}
```

JSON Request ví dụ

```
{  
  "userId": 1,  
  "title": "Hướng d[]n Spring Boot từ đ[]u",  
  "content": "Spring Boot giúp xây dựng REST API nhanh chóng...",  
  "imageUrl": "https://cdn.example.com/img/spring.jpg",  
  "status": "PUBLISHED"  
}
```

Response thành công (201 Created)

```
{  
  "success": true,  
  "message": "Tạo bài vi[]t thành công",  
  "statusCode": 201,  
  "data": {  
    "userId": 1,  
    "title": "Hướng d[]n Spring Boot từ đ[]u",  
    "content": "Spring Boot giúp xây dựng REST API nhanh chóng...",  
    "imageUrl": "https://cdn.example.com/img/spring.jpg",  
    "status": "PUBLISHED"  
  }  
}
```

```
"id": 42,  
"userId": 1,  
"title": "Hướng dẫn Spring Boot từ đầu",  
"content": "Spring Boot giúp xây dựng REST API nhanh chóng...",  
"imageUrl": "https://cdn.example.com/img/spring.jpg",  
"status": "PUBLISHED",  
"viewCount": 0,  
"createdAt": "2026-03-06T10:00:00"  
}  
}
```

Validate lỗi (400 Bad Request) - ví dụ

```
{  
  "success": false,  
  "message": "Dữ liệu không hợp lệ",  
  "statusCode": 400,  
  "data": {  
    "title": "Tiêu đề từ 5 đến 255 ký tự",  
    "content": "Nội dung không được để trống"  
  }  
}
```

Source Code - PostController.java

```
@PostMapping  
public ResponseEntity<ApiResponse<PostResponse>> createPost(  
    @Valid @RequestBody CreatePostRequest request) {  
    PostResponse data = postService.createPost(request);  
    return ResponseEntity.status(201)  
        .body(ApiResponse.success("Tạo bài viết thành công", data));  
}
```

Source Code - PostServiceImpl.java

```
@Override  
public PostResponse createPost(CreatePostRequest req) {  
    // 1. Kiểm tra user tồn tại  
    User user = userRepository.findById(req.getUserId())  
        .orElseThrow(() -> new ResourceNotFoundException(  
            "Không tìm thấy user với id = " + req.getUserId()));  
    // 2. Kiểm tra user có trạng thái ACTIVE  
    if (user.getStatus() != UserStatus.ACTIVE) {  
        throw new BadRequestException("Tài khoản không được phép đăng bài");  
    }  
    // 3. Tạo entity và lưu
```



```
Post post = new Post();
post.setUser(user);
post.setTitle(req.getTitle());
post.setContent(req.getContent());
post.setImageUrl(req.getImageUrl());
post.setStatus(PostStatus.valueOf(req.getStatus()));
post.setViewCount(0);

Post saved = postRepository.save(post);
return postMapper.toResponse(saved);
}
```

5. Yêu cầu các API còn lại (từ Dễ → Khó)

Mỗi API bên dưới liệt kê đầy đủ: endpoint, method, request (params/body), response mong đợi, validate, xử lý lỗi và gợi ý cài đặt.

5.1 Nhóm User

GET	/api/v1/users	Dễ
-----	---------------	----

Lấy danh sách tất cả người dùng, hỗ trợ phân trang và lọc theo status.

□ Request: Query params: page (default=0), size (default=10), sort, status (optional: ACTIVE|INACTIVE|BANNED)

□ Response: 200 OK – Page gồm id, username, fullName, avatarUrl, status, createdAt.

□ **Validate & Lỗi:**

- page ≥ 0 , size trong [1..100]
- sort chỉ nhận: createdAt, username, fullName

□ Dùng Pageable của Spring Data JPA, truyền vào Repository.findAll(spec, pageable).

POST	/api/v1/users	Dễ
------	---------------	----

Tạo tài khoản người dùng mới.

□ Request: Body JSON: username*, email*, fullName*, avatarUrl (optional), bio (optional)

□ Response: 201 Created – UserResponse đầy đủ. 400 nếu validate lỗi. 409 nếu username/email đã tồn tại.

□ **Validate & Lỗi:**

- username: 3-50 ký tự, chỉ chữ thường/số/dấu gạch dưới, không khoảng trắng
- email: đúng định dạng email, tối đa 100 ký tự
- fullName: 2-100 ký tự, không được trống
- Kiểm tra UNIQUE: username và email không được trùng → 409 Conflict

□ Service phải gọi existsByUsername và existsByEmail trước khi save.

PUT	/api/v1/users/{id}	Dễ
-----	--------------------	----

Cập nhật toàn bộ thông tin người dùng (fullName, bio, avatarUrl).

□ Request: Path: id. Body JSON: fullName*, avatarUrl (optional), bio (optional)

□ Response: 200 OK – UserResponse cập nhật mới. 404 nếu không tìm thấy user.

□ **Validate & Lỗi:**

- fullName: bắt buộc, 2-100 ký tự
- avatarUrl nếu có phải đúng định dạng URL
- 404 nếu id không tồn tại

□ Chú ý: không cho phép cập nhật username và email ở endpoint này.

PATCH`/api/v1/users/{id}/status`**Dễ**

Thay đổi trạng thái tài khoản người dùng.

□ Request: Path: id. Body JSON: { "status": "ACTIVE" | "INACTIVE" | "BANNED" }

□ Response: 200 OK – { "id": 1, "status": "BANNED" }. 404 nếu không tìm thấy.

□ **Validate & Lỗi:**

- status chỉ nhận một trong 3 giá trị: ACTIVE, INACTIVE, BANNED
- 404 nếu user không tồn tại

DELETE`/api/v1/users/{id}`**Dễ**

Xóa người dùng. Do có ON DELETE CASCADE nên sẽ xóa cả posts, likes, comments liên quan.

□ Request: Path: id (bắt buộc)

□ Response: 204 No Content nếu thành công. 404 nếu không tìm thấy.

□ **Validate & Lỗi:**

- 404 nếu id không tồn tại
- Cần nhắc soft delete thay vì hard delete trong production

□ *Hard delete → dùng deleteById. Soft delete → thêm cột deleted_at vào bảng users.*

5.2 Nhóm Post

GET`/api/v1/posts`**Dễ**

Danh sách bài viết đã công khai (status=PUBLISHED), hỗ trợ lọc theo userId và phân trang.

□ Request: Query params: page, size, sort, userId (optional), keyword (optional)

□ Response: 200 OK – Page: id, title, content(tối đa 200 ký tự), imageUrl, status, viewCount, userId, createdAt.

□ **Validate & Lỗi:**

- Chỉ trả về bài status=PUBLISHED (trừ khi filter khác)
- keyword tìm trong title (LIKE %keyword%)

GET`/api/v1/posts/{id}`**Dễ**

Lấy chi tiết một bài viết và tăng viewCount lên 1 mỗi lần gọi.

□ Request: Path: id

□ Response: 200 OK – PostResponse đầy đủ. 404 nếu không tìm thấy hoặc bị xóa.

□ **Validate & Lỗi:**

- 404 nếu post không tồn tại hoặc status = DELETED
- Dùng @Transactional khi tăng viewCount để tránh race condition

PUT`/api/v1/posts/{id}`**Dễ**

Cập nhật bài viết (chỉ người tạo mới được sửa – kiểm tra userId trong body).

□ Request: Path: id. Body: userId*, title*, content*, imageUrl (opt), status (opt)

□ Response: 200 OK – PostResponse. 404 nếu không tìm. 403 nếu userId không phải chủ bài.

□ **Validate & Lỗi:**

- userId trong body phải khớp với user_id của post → 403 Forbidden nếu không khớp
- title: 5–255 ký tự
- content: tối thiểu 10 ký tự
- status chỉ nhận PUBLISHED hoặc DRAFT (không thể set DELETED qua endpoint này)

DELETE`/api/v1/posts/{id}`**Dễ**

Xóa mềm bài viết (soft delete: đặt status = DELETED).

□ Request: Path: id. Body hoặc Query param: userId (để xác định chủ bài)

□ Response: 200 OK – { "message": "Xóa bài viết thành công" }. 404 hoặc 403.

□ **Validate & Lỗi:**

- 403 nếu userId không phải chủ bài viết
- 404 nếu bài viết không tồn tại

□ *Soft delete: `postRepository.updateStatus(id, PostStatus.DELETED)` bằng `@Modifying @Query`.*

GET`/api/v1/posts/search`**Trung bình**

Tìm kiếm bài viết theo keyword trong title hoặc content.

□ Request: Query params: keyword* (bắt buộc, tối thiểu 2 ký tự), page, size

□ Response: 200 OK – Page. Chỉ bài PUBLISHED. Sắp xếp theo độ liên quan (createdAt desc mặc định).

□ **Validate & Lỗi:**

- keyword tối thiểu 2 ký tự
- keyword tối đa 100 ký tự

□ *Repository: `findByStatusAndTitleContainingOrContentContaining(...)` hoặc dùng `@Query JPQL`.*

5.3 Nhóm Like

POST

/api/v1/likes

Trung bình

Toggle like/unlike: nếu chưa like thì like, nếu đã like thì unlike (xóa).

□ Request: Body JSON: { "userId": 1, "postId": 5 }

□ Response: 200 OK – { "liked": true, "likeCount": 12 } hoặc { "liked": false, "likeCount": 11 }.

□ **Validate & Lỗi:**

- userId và postId phải tồn tại → 404 nếu không tìm thấy
- post phải có status = PUBLISHED → 400 nếu đang DRAFT hoặc DELETED
- Bắt DataIntegrityViolationException để xử lý race condition UNIQUE constraint

□ Service kiểm tra bằng existsByUserIdAndPostId. Dùng @Transactional để đảm bảo atomic.

GET

/api/v1/posts/{id}/likes

Trung bình

Lấy danh sách người dùng đã like bài viết (JOIN likes và users).

□ Request: Path: id (postId). Query params: page, size

□ Response: 200 OK – Page (id, username, fullName, avatarUrl) của những người đã like.

□ **Validate & Lỗi:**

- 404 nếu post không tồn tại
- Chỉ trả về like của bài PUBLISHED

□ Repository: @Query("SELECT u FROM User u JOIN Like l ON l.user=u WHERE l.post.id=:postId")

5.4 Nhóm Comment

GET

/api/v1/posts/{id}/comments

Trung bình

Lấy danh sách comment gốc (parent_id IS NULL) của một bài viết, kèm replyCount.

□ Request: Path: id (postId). Query params: page, size

□ Response: 200 OK – Page: id, content, userId, username, avatarUrl, replyCount, createdAt.

□ **Validate & Lỗi:**

- 404 nếu postId không tồn tại
- Chỉ lấy comment có status = VISIBLE

□ Cần LEFT JOIN hoặc subquery để đếm replyCount. Sắp xếp theo createdAt ASC mặc định.

POST`/api/v1/comments`**Trung bình**

Thêm comment mới. Nếu có parentId thì là reply của comment đó.

□ Request: Body JSON: { "userId":1, "postId":5, "content":"Bài hay!", "parentId": null }

□ Response: 201 Created – CommentResponse đầy đủ. 404 nếu user/post/parentComment không tồn tại.

□ **Validate & Lỗi:**

- content: 1-1000 ký tự, không được trống
- parentId nếu có phải thuộc cùng postId → 400 nếu parentComment.postId != postId
- post phải PUBLISHED → 400 nếu không phải
- user phải ACTIVE → 400 nếu bị BANNED

PUT`/api/v1/comments/{id}`**Trung bình**

Chỉnh sửa nội dung comment (chỉ chủ comment mới sửa được).

□ Request: Path: id. Body: { "userId": 1, "content": "Nội dung mới" }

□ Response: 200 OK – CommentResponse cập nhật. 403 nếu không phải chủ. 404 nếu không tìm thấy.

□ **Validate & Lỗi:**

- content: 1-1000 ký tự
- 403 nếu userId không phải người tạo comment

DELETE`/api/v1/comments/{id}`**Trung bình**

Xóa mềm comment (status = DELETED). Các reply con vẫn giữ nguyên.

□ Request: Path: id. Query param: userId (xác định chủ comment)

□ Response: 200 OK. 403 nếu không phải chủ. 404 nếu không tìm thấy.

□ **Validate & Lỗi:**

- 403 nếu userId không phải chủ comment

GET`/api/v1/comments/{id}/replies`**Trung bình**

Lấy danh sách reply trực tiếp của một comment (parent_id = id).

□ Request: Path: id (commentId). Query params: page, size

□ Response: 200 OK – Page. Chỉ status = VISIBLE.

□ **Validate & Lỗi:**

- 404 nếu comment cha không tồn tại

5.5 API nâng cao - JOIN nhiều bảng (Từ Trung bình → Khó)

GET

/api/v1/users/{id}/posts

Trung bình

Lấy tất cả bài viết của một user, kèm like count và comment count mỗi bài.

□ Request: Path: id (userId). Query params: page, size, status (optional)

□ Response: 200 OK – Page: id, title, status, viewCount, likeCount, commentCount, createdAt.

□ **Validate & Lỗi:**

- 404 nếu userId không tồn tại

□ JPQL: *SELECT p, COUNT(DISTINCT l), COUNT(DISTINCT c) FROM Post p LEFT JOIN Like l ON l.post=p LEFT JOIN Comment c ON c.post=p WHERE p.user.id=:userId GROUP BY p.id*

GET

/api/v1/posts/top-liked

Khó

Top N bài viết có nhiều lượt like nhất, sắp xếp giảm dần.

□ Request: Query params: top (default=10, max=50)

□ Response: 200 OK – List: rank, id, title, authorName, likeCount, commentCount.

□ **Validate & Lỗi:**

- top trong [1..50] → 400 nếu ngoài khoảng

□ Dùng *GROUP BY + ORDER BY + LIMIT*. Kết hợp *JOIN posts, users, likes, comments*.

GET

/api/v1/posts/{id}/detail

Khó

Chi tiết đầy đủ bài viết: thông tin tác giả + tổng like + tổng comment + 5 comment mới nhất.

□ Request: Path: id

□ Response: 200 OK – PostDetailResponse: { post, author: UserSummary, likeCount, commentCount, recentComments: [...] }

□ **Validate & Lỗi:**

- 404 nếu post không tồn tại hoặc DELETED

□ Cần nhiều query hoặc 1 native query phức tạp. Nên dùng *DTO projection* để tối ưu.

GET

/api/v1/users/{id}/stats

Khó

Thống kê hoạt động của một user: số bài đăng, tổng like nhận, tổng comment nhận, tổng view.

□ Request: Path: id

□ Response: 200 OK – UserStatsResponse: { userId, postCount, totalLikesReceived, totalCommentsReceived, totalViews }

□ **Validate & Lỗi:**

- 404 nếu userId không tồn tại

□ Cần *aggregate query*: *SUM(view_count), COUNT(likes JOIN posts WHERE posts.user_id=id), COUNT(comments JOIN posts ...)*.

GET`/api/v1/feed`**Khó**

Feed tổng hợp: các bài viết mới nhất của tất cả user, mỗi bài kèm thông tin tác giả (avatar, username) và like count.

□ Request: Query params: page, size (default=20)

□ Response: 200 OK – Page: { postId, title, authorUsername, authorAvatar, likeCount, commentCount, createdAt }

□ **Validate & Lỗi:**

- size tối đa 50 mỗi trang

□ *JOIN posts p, users u ON p.user_id=u.id, LEFT JOIN likes l GROUP BY p.id ORDER BY p.created_at DESC.*

GET`/api/v1/comments/tree/{postId}`**Khó**

Trả về cây comment dạng lồng nhau (nested) cho một bài viết (root comments + replies của từng comment).

□ Request: Path: postId. Query param: maxDepth (default=2)

□ Response: 200 OK – List: mỗi node gồm { comment, children: [CommentTreeNode] }

□ **Validate & Lỗi:**

- 404 nếu postId không tồn tại
- maxDepth trong [1..3]

□ *Lấy toàn bộ comment của postId, sau đó build tree trong Java (Map>). Không dùng recursive query để tránh N+1.*

GET`/api/v1/posts/trending`**Khó**

Trending: tính điểm = (like_count * 2) + comment_count trong 7 ngày gần nhất. Trả về top N bài có điểm cao nhất.

□ Request: Query params: top (default=10), days (default=7, max=30)

□ Response: 200 OK – List: { rank, postId, title, authorName, score, likeCount, commentCount }

□ **Validate & Lỗi:**

- top trong [1..50]
- days trong [1..30]

□ *Native SQL hoặc JPQL với subquery COUNT likes trong khoảng thời gian. Sắp xếp theo score DESC. Cân nhắc cache kết quả với @Cacheable (Redis).*

6. Quy ước chung & Validate

6.1 HTTP Status Code chuẩn

HTTP Status	Ý nghĩa
200 OK	Lấy dữ liệu thành công, cập nhật thành công
201 Created	Tạo mới thành công (POST)
204 No Content	Xóa thành công (không có body trả về)
400 Bad Request	Dữ liệu đầu vào không hợp lệ (validate fail, logic lỗi)
403 Forbidden	Không có quyền thực hiện (không phải chủ tài nguyên)
404 Not Found	Không tìm thấy tài nguyên theo id
409 Conflict	Tài nguyên đã tồn tại (duplicate username, email)
500 Internal	Lỗi server không mong đợi

6.2 Annotation Validate thường dùng

Annotation	Mô tả
@NotNull	Trường không được null (kể cả chuỗi rỗng)
@NotBlank	Chuỗi không được null, rỗng, hay chỉ có khoảng trắng
@Size(min,max)	Độ dài chuỗi hoặc collection trong khoảng [min, max]
@Min / @Max	Giá trị số tối thiểu / tối đa
@Positive	Số nguyên phải > 0 (dùng cho id)
@Email	Chuỗi phải đúng định dạng email
@URL	Chuỗi phải đúng định dạng URL (hibernate-validator)
@Pattern(regexp)	Chuỗi phải khớp với biểu thức chính quy
@Valid	Kích hoạt validate lồng nhau (nested DTO)

6.3 Các lưu ý khi triển khai

- Phân trang mặc định: page=0, size=10, sort=createdAt,desc. Dùng Pageable trong controller và truyền vào service/repository.
- Soft Delete: Không xóa dữ liệu khỏi DB. Đặt status=DELETED. Mọi query lấy danh sách cần filter status != DELETED.
- viewCount concurrency: Dùng @Modifying @Query("UPDATE Post p SET p.viewCount = p.viewCount + 1 WHERE p.id=:id") thay vì load entity rồi set để tránh race condition.

- Toggle Like atomic: Luôn dùng @Transactional. Dùng existsByUserIdAndPostId để kiểm tra, sau đó save hoặc delete.
- N+1 Query: Với các API fetch list, dùng JOIN FETCH hoặc @EntityGraph để tránh N+1. Ví dụ: fetch posts kèm user author.
- DTO Projection: Với API phức tạp (stats, trending), dùng interface-based projection hoặc class-based DTO thay vì fetch toàn bộ entity.
- Naming Convention: camelCase cho JSON fields. snake_case cho cột DB. PascalCase cho class Java.
- Exception tập trung: Mọi exception đều được bắt tại GlobalExceptionHandler. Không để lộ stack trace ra client trong môi trường production.

6.4 Dependencies gợi ý (pom.xml)

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- mysql-connector-j
- lombok
- mapstruct (optional – auto mapper)
- springdoc-openapi-starter-webmvc-ui (Swagger UI)

6.5 application.yml mẫu

```
spring:
datasource:
url: jdbc:mysql://localhost:3306/social_app?useUnicode=true&characterEncoding=UTF-8
username: root
password: yourpassword
driver-class-name: com.mysql.cj.jdbc.Driver
jpa:
hibernate:
ddl-auto: validate # dùng Flyway/Liquibase trong production
show-sql: true
properties:
hibernate.format_sql: true
hibernate.dialect: org.hibernate.dialect.MySQL8Dialect
server:
port: 8080
servlet:
context-path: /api/v1
```
